

技术参考

TDS Toolkit 编写协议解码器

适用于 TDS 500、TDS 600 和 TDS 700 系列数字化示波器

文档 TDSTK-DEC-01
Copyright © 2026 Jared Cabot。MIT 许可证。

TDSTK-DEC-01

Copyright © 2026 Jared Cabot。保留所有权利。

本文档描述 TDS Toolkit
的解码器插件接口。本出版物中的信息取代此前发布的所有材料中的相应信息。

TDS Toolkit 是主机软件，并非 Tektronix 产品。Tektronix 和 TDS
仅用于指称本软件所支持的仪器。

本软件依据 MIT
许可证发布。本软件按“原样”提供，不作任何明示或暗示的保证。全文以 LICENSE
随程序一同提供。

目录

	相关手册	iv
1	解码器模型	1-1
	什么是解码器	1-1
	一个文件，一种协议	1-1
	约定	1-1
	发现	1-1
	文件的位置	1-1
2	Signal 对象	2-1
	解码得到什么	2-1
	采样与时序	2-1
	电平，而非采样	2-1
	Signal 从何而来	2-2
3	Decoder 约定	3-1
	类属性	3-1
	四个属性	3-1
	角色与源	3-1
	方法	3-1
	decode	3-1
	参数	3-2
	Param 类型	3-2
4	帧与渲染	4-1
	Frame 对象	4-1
	一个已解码事件	4-1
	状态	4-1
	渲染结果	4-2
	四个小方法	4-2
	颜色	4-2

5	构建解码器	5-1
	一个完整示例	5-1
	协议	5-1
	步骤	5-1
	代码	5-1
	导入与属性	5-1
	恢复比特	5-1
	组装字	5-2
	渲染	5-2
	串行捷径	5-3
	不要重新实现比特采样	5-3
6	安装与测试	6-1
	安装	6-1
	放入并打开	6-1
	测试	6-1
	用合成 Signal 驱动它	6-1
	测试每个解码器	6-1
	检查清单	6-2
	在依赖它之前	6-2

表格目录

表 1 – 1: 解码器子系统的文件	1 – 2
表 2 – 1: Signal 的成员	2 – 2
表 3 – 1: Decoder 类属性	3 – 1
表 3 – 2: Decoder 方法	3 – 2
表 3 – 3: Param 的种类	3 – 2
表 4 – 1: Frame 字段	4 – 1
表 4 – 2: Frame 状态位	4 – 2

前言

本手册介绍如何为 TDS Toolkit 添加协议解码器。一个解码器就是一个 Python 文件。它读取捕获的波形并返回一系列已解码事件；本程序会将其列出、映射其输入、运行它并绘制结果，无需任何额外代码。

本手册假定读者是熟练使用 Python 的程序员。无需了解本程序的其余部分：整个接口就是一个基类和三个小型数据类型，全部位于模块 `tds_decode.py` 中。

请对照随程序提供的两个文件阅读本手册：`decoders/_template.py`，即可复制的骨架，以及 `decoders/README.txt`，即下文的简短版本。

相关手册

接口，以及应对照本手册阅读的文件：

- `decoders/_template.py` - 可复制的骨架。
- `decoders/README.txt` - 下文的简短版本。
- `tds_decode.py` - 引擎：Decoder 基类以及 Signal、Frame 和 Param 类型。

解码器模型

什么是解码器

一个文件，一种协议

解码器是 decoders 文件夹中的单个 Python 文件，它定义 Decoder 的一个子类。每个文件对应一种协议。“波形”选项卡会列出它找到的每个解码器，因此添加一种协议只需一个文件，程序中的其余部分无需改动。解码完全在“波形”选项卡已持有的捕获波形上运行：从仪器读取的通道，或从文件加载的已存储参考。解码器中的任何部分都不触及仪器。这正是本设计的用意所在。它可在任何 TDS 上工作，包括固件无法运行自身总线解码应用的基础型号，因为解码是在捕获之后、在主机上、对采样数据进行的。

约定

整个接口就是 Decoder 基类。解码器设置四个类属性并实现一个方法。其余三个方法都有可用的默认实现，仅在需要在表格和绘图中显示本协议自身字段时才需覆盖。解码器所处理的三个数据类型 Signal、Frame 和 Param 将在后续各章中说明。

发现

在启动时，以及每当打开“波形”选项卡时，本程序都会扫描 decoders 文件夹中的 *.py 文件，将每个文件单独导入，并实例化它找到的每个 Decoder 子类。文件名以下划线开头的文件会被当作辅助文件而跳过，_template.py 正是借此保持为模板，永远不会作为空协议出现在列表中。导入是防御性的。无法导入的文件——无论是因语法错误还是错误的引用——都会被跳过，并在程序日志中写入一条说明，而不会连累整个列表。未完成的解码器绝不会遮蔽可用的解码器。当两个文件夹提供 NAME 相同的解码器时，后面的文件夹优先，因此本地编写的副本会覆盖随附的解码器。

文件的位置

随程序提供的解码器位于安装目录内的 decoders 文件夹中。一份可写副本保存在设置文件 tdstoolkit.json 旁边，并在每次运行时从随附的集合中补齐，这与掩模库完全一样。在后续版本中新增的解码器会被复制过来，而不会覆盖本地编写的解码器；被删除的随附解码器会在下次运行时恢复，因为随附集合本应存在。将一个新文件放入可写文件夹，它便会在下次打开该选项卡时出现在列表中。无需重新构建。

表 1 – 1: 解码器子系统的文件

文件	角色
tds_decode.py	引擎：Decoder 基类，Signal、Frame 和 Param 类型，状态常量，共享的串行辅助函数，以及发现机制。解码器从中导入，但不修改它。
decoders/_template.py	骨架。复制它以开始编写新的解码器。
decoders/README.txt	简短版说明，以及各示例解码器的概览。
decoders/.py	每个对应一种协议：uart.py、i2c.py、spi.py、can.py 等。请阅读与所添加协议最接近的那个。

Signal 对象

解码得到什么

	<code>decode()</code> 方法会为协议所需的每个输入接收一个 <code>Signal</code>。Signal 是一条经过精简、只保留解码器所需内容的捕获记录：整数采样值，以及把采样索引换算为时间的时序信息。它不携带电压，也不含显示缩放。解码器将采样与判决电平比较，从不需要知道电压，因此原始数字化计数值不经缩放直接传入，传递一条二十五万点的记录也几乎没有开销。
采样与时序	判决电平、迟滞死区和信号跨度都在调用 <code>decode()</code> 之前算出。解码器不自行计算判决电平。该电平取记录极值的中点；迟滞取跨度的八分之一，足够宽以抑制边沿振铃，又不至于让缓慢的边沿无法越过它。这些值只供读取，不应设置。
电平，而非采样	对大多数协议而言，记录的有用形式并非原始采样，而是每个采样对应一个干净的逻辑电平。<code>digitize()</code> 方法返回的正是这个：为每个采样给出 0 或 1，并已应用记录自身的迟滞，从而边沿振铃不会被读成多次跨越。调用一次并遍历其结果即可。若某协议需要原始计数值——例如需要两个判决电平的双极性线路——则改为直接读取 <code>sig.data</code>。

表 2 – 1: Signal 的成员

成员	含义
sig.data	整数采样值，一个可索引的原始数字化计数值序列。
sig.n	采样点数，与 len(sig.data) 相同。
sig.dt	每个采样的秒数。
sig.t0	第一个采样的时间，以相对于触发的秒数表示。
sig.first_index	第一个采样在记录中的偏移。将其与局部采样索引相加即得记录中的绝对位置。
sig.threshold	判决电平，以计数值表示。由框架设置。
sig.hysteresis	判决电平周围的死区，以计数值表示。由框架设置。
sig.span	记录中最高采样与最低采样之差。由框架设置；跨度接近零意味着源平坦、无信号。
sig.time_at(i)	局部采样索引 i 处的时间（秒），即 $t0 + i \times dt$ 。
sig.digitize()	一个新的 bytearray，每个采样对应一个 0/1 电平，并已应用迟滞。
sig.auto_threshold()	重新计算判决电平、迟滞和跨度。框架已调用它；仅在手工构建 Signal 时才需调用（见第 6 章）。

说明. Signal 携带的是原始计数值，而非电压。请与 sig.threshold 比较，切勿与某个电压比较。在某一垂直刻度下读取正确的解码器，在其他任何刻度下也都读取正确，因为判决电平随记录一同变化。

Signal 从何而来

decode() 收到的 Signal 由本程序而非解码器构建，它通过类方法 Signal.from_waveform() 从捕获的 Waveform 生成。该方法从波形前导中读取整数电平、采样间隔和触发偏移，构造出 Signal，并在交给解码器之前对其调用 auto_threshold()。仅在编写测试时才需手工构建 Signal，这在第 6 章中介绍。

Decoder 约定

类属性

解码器子类设置四个类属性。NAME 和 DESCRIPTION 用于在协议列表中为其命名。SOURCES 声明它所需的波形输入。PARAMS 声明选项卡所提供的设置。其余一切均为继承而来。

表 3 – 1: Decoder 类属性

属性	含义
NAME	协议名称。显示在列表中，并用作绘图标签的标题。将其留空或设为 "?", 加载器便不会列出该类；模板正是借此避开列表。
DESCRIPTION	用于列表第二列的一行文字。
SOURCES	一个按顺序排列的角色名称列表。一个角色即单线协议；两个或更多角色则各自映射一个捕获的源。末尾的 "?" 标记一个可不映射的可选角色。
PARAMS	一个 Param 对象列表，每个对应一个控件。可为空。
COLOUR	正常帧的人字形颜色，形如 "#rrggbb"。解码器自行拥有其颜色；本程序对解码器没有颜色设置。设置此属性即可为整个协议指定一种颜色。
FAULT_COLOUR	故障帧的人字形颜色。默认为红色，因此无论协议使用何种颜色，错误都很醒目。

角色与源 SOURCES 既是协议所需波形的数量，也是它们的名称。单线协议声明一个角色，按惯例为 "Line"。像 I2C 这样的双线协议声明 ["SDA", "SCL"]；选项卡为每个角色显示一个源选择器，decode() 会为每个角色接收一个 Signal。末尾的问号（如 SPI 的 ["CLK", "MOSI", "MISO?", "CS?"]）标记操作员可不映射的角色；此时其 Signal 不会出现在 decode() 收到的字典中，需由 decode() 代码检查其是否存在。

方法

decode decode(self, signals, params) 是唯一必需实现的方法。signals 是一个字典，将每个角色名称（不含末尾的 "?"）映射到其 Signal；未映射的可选角色则不在其中。params 是一个字典，将每个 Param 的 key 映射到操作员所选的值。该方法返回一个二元组：一个 Frame 对象列表，以及一行注释字符串。

该注释显示在结果表上方。用它报告解码推断出的信息，例如检测到的比特率、每比特采样数和找到的帧数，或者在无法读取记录时说明原因。失败时，返回空的帧列表，以及一条用一句话解释问题的注释。

表 3 – 2: Decoder 方法

方法	返回
decode(signals, params)	(frames, note)。解码本身。必需。
columns()	“ 时间 ” 列之后的表头，形式为 (heading, width_in_pixels) 对的列表。覆盖它以命名协议自身的字段。
row(frame, ascii)	一个帧的表格单元格，在时间之后与 columns() 对应。覆盖它以格式化协议自身的字段。
label(frame, ascii)	用于绘图上人字形的最短有用字符串。覆盖它以保持简短。
colour(frame)	一个帧的人字形颜色。正常帧默认为 COLOUR，故障帧默认为 FAULT_COLOUR。覆盖它可按 frame.kind 分支，为每个人字形单独着色。

传给 row() 和 label() 的 ascii 参数是选项卡上 Hex / ASCII 切换的状态。在有意义之处遵从它——将一个字节显示为字符或显示为两位十六进制数——在无意义之处则忽略它。

参数

PARAMS 列表中的每个 Param 都会成为选项卡上的一个控件，因此协议无需任何用户界面代码即可添加自己的选项。其构造函数为 Param(key, label, kind, choices, default, lo, hi, unit, note)。key 是 decode() 读取的字典键；label 是操作员看到的说明文字；kind 是三个字符串之一。

表 3 – 3: Param 的种类

kind	控件与字段
"choice"	下拉框。choices 是 (value, label) 对的列表；value 是传入 params 的内容，label 是所显示的内容。default 为其中某个 value。
"int"	整数输入框。lo 和 hi 为其上下界；unit 是标题后缀，例如 "bits"。按惯例，默认值 0 表示 “ 自动 ”，note 中应予以说明。
"bool"	复选框。default 为 True 或 False。

Param 类型 未提供 default 时，choice 回退为第一个选项，bool 回退为 False，int 回退为 0。可选的 note 会作为提示显示在控件旁边。

帧与渲染

Frame 对象

一个已解码事件

Frame 是一个已解码事件：一个 UART 字符、一个 I2C 字节、一个 SPI 字、一次总线状态。该类型是扁平的，为每种协议所共用。它携带一个主数据、一个可选的次数据、一个长度、一个状态和一个种类。这些字段的含义由解码器自行决定，因为只有生成某帧的解码器才会渲染它，但其结构是固定的。

用关键字参数构造 Frame；每个字段都有默认值。两个索引字段最为重要，也最容易出错。index 和 end_index 是记录中的绝对采样位置：绘图会在两者之间画出入字形框。构造它们的方式是 sig.first_index 加上局部采样索引，正如时间取自对局部索引调用的 sig.time_at()。

表 4 – 1: Frame 字段

字段	含义
time	事件的起始时间（秒），取自 sig.time_at(local_index)。
index	绝对起始采样：sig.first_index + 局部起始。
end_index	绝对结束采样：sig.first_index + 局部结束。方框从 index 跨到 end_index。
value	主数据：字节、字或测得的值。
tag	可选的次数据：I2C 地址、SPI MISO 字或测得的频率。未使用时为 0。
length	数据的长度，以协议所计的单位表示。默认为 1。
status	状态常量的按位或；帧正常时为 OK（零）。
kind	一个自由字符串，例如 "byte"、"addr"、"data"、"start" 或 "stop"，row() 和 label() 据此分支，以便按各自方式渲染帧的每一部分。

状态

status 是一个位掩码。正常帧为 OK，即零。有问题的帧将表 4-2 中的一个或多个常量按位或在一起。框架会把状态非零的任何帧画成错误——红色人字形以及标签中前置的感叹号——解码器除了置位之外无需做任何事。

表 4 – 2: Frame 状态位

常量	值	含义
OK	0	正常；无错误。
FRAMING_ERROR	1	帧未按其格式要求收尾，例如 UART 停止位不是标记电平。
PARITY_ERROR	2	奇偶校验位与数据不符。
TRUNCATED	4	记录在帧中途结束。
ACK_ERROR	8	本应得到应答的字节未获应答。

请连同 Decoder、Frame 和 Param 一起，从 tds_decode 导入所用的这些常量。不要定义这五个之外的任何状态值；渲染器只理解以上五个。

渲染结果

四个小方法

基类将帧渲染为一个普通字节：十六进制、ASCII 和一个状态字。覆盖 columns()、row() 和 label() 即可改用协议自身的表述。columns() 返回“时间”列之后的表头，每个都带一个像素宽度。row() 按相同顺序为每个表头返回一个单元格。label() 返回画在波形本身之上的人字形的最短有用字符串。

若某协议的帧带有不同的种类，就在这些方法中按 frame.kind 分支，使 START、地址字节和数据字节在表中和绘图上都能按其自身含义呈现。请保持 label() 简短：它绘制在波形之上的一个小框内，因此一到三个字符是合适的长度。

颜色

第四个方法 colour(frame) 设置人字形颜色，且完全由解码器掌控：本程序并未为解码器提供颜色设置。默认将正常帧画成类的 COLOUR、将故障帧画成 FAULT_COLOUR，因此设置 COLOUR 便为整个协议赋予单一颜色。覆盖 colour() 并按 frame.kind 分支，即可为每个人字形着以不同颜色，使同一条线的地址、数据和帧结构各以自己的颜色呈现。返回一个 "#rrggb"；对于解码器自身不着色的种类，回退到基类。

构建解码器

一个完整示例

协议 本章从模板出发构建一个完整的解码器。示例协议是一根自时钟的单线，每个脉冲携带一个比特：线路上升，保持高电平达比特周期的一部分，然后下降；短高电平为 0，长高电平为 1，由高电平时间是否超过周期的一半来判定。比特按选定的字长和比特序被组装成字。WS2812 像素灯带和 NEC 红外遥控都属于这一类协议，而它不在内置协议之中，因此它是一个真正的新解码器，而非对某个已有解码器的复制。

- 步骤**
1. 复制模板：将 `decoders/_template.py` 复制为 `decoders/pulsebits.py`。
 2. 命名协议：重命名该类并设置 `NAME`、`DESCRIPTION` 和 `SOURCES`。
 3. 声明设置：在 `PARAMS` 中填入字长和比特序控件。
 4. 编写解码：从电平序列中恢复比特，然后将它们组装成字。
 5. 渲染字段：覆盖 `columns()`、`row()` 和 `label()`，以十六进制和二进制显示该字。
 6. 测试它：用合成 `Signal` 驱动解码器并断言结果，如第 6 章所示。

代码

导入与属性 文件头仅从 `tds_decode` 导入，命名该类，并声明选项卡将提供的两个输入：字长和比特序。`SOURCES` 只含一个角色，因此选项卡显示单个源选择器。

```
from tds_decode import Decoder, Frame, Param
from tds_decode import OK, TRUNCATED

class PulseBitsDecoder(Decoder):

    NAME = "Pulse-width bits"
    DESCRIPTION = "Self-clocked pulse-width line, one wire"

    SOURCES = ["Line"]
    PARAMS = [
        Param("bits", "Word size", "int", default=8,
              lo=1, hi=32, unit="bits"),
        Param("order", "Bit order", "choice",
              choices=[("msb", "MSB first"),
                       ("lsb", "LSB first")],
              default="msb"),
    ]
```

恢复比特

decode() 取用这唯一的 Signal，防范平坦的源，并用 digitize() 将记录化简为每个采样一个 0/1 电平。随后它遍历该电平序列。每个比特始于一个上升沿；高采样的连续段即脉冲，其后低采样的连续段延续到下一个上升沿，两者合起来即比特周期。将高电平时间与周期相比较，即可判定该比特。

```
def decode(self, signals, params):
    sig = signals["Line"]
    if sig.span < 8:
        return [], "no signal on this source"
    level = sig.digitize()
    n = len(level)
    word_bits = max(1, int(params.get("bits") or 8))
    msb = params.get("order", "msb") == "msb"

    bits = []          # (value, start_index, end_index)
    i = 1
    while i < n:
        if level[i] and not level[i - 1]: # a rising edge
            start = i
            while i < n and level[i]:      # the high pulse
                i += 1
            high = i - start
            while i < n and not level[i]:  # low, to next rise
                i += 1
            period = i - start
            if period > 0:
                bit = 1 if high * 2 > period else 0
                bits.append((bit, start, i))
        else:
            i += 1
```

组装字

然后按 word_bits 个一组取用这些比特，依参数取最高位在先或最低位在先。因记录结束而不足一组的组仍会被输出，其状态标记为 TRUNCATED，以便渲染器将其画成错误。每个 Frame 的时间取自第一个比特，跨度从第一个比特的起始到最后一个比特的结束。注释会报告各项总数。

```
frames = []
for base in range(0, len(bits), word_bits):
    chunk = bits[base:base + word_bits]
    value = 0
    for pos, (bit, _s, _e) in enumerate(chunk):
        shift = (word_bits - 1 - pos) if msb else pos
        value |= bit << shift
    start_local = chunk[0][1]
    end_local = chunk[-1][2]
    ok = len(chunk) == word_bits
    frames.append(Frame(
        time=sig.time_at(start_local),
        index=sig.first_index + start_local,
        end_index=sig.first_index + end_local,
        value=value, length=len(chunk),
        status=OK if ok else TRUNCATED,
        kind="data"))
return frames, "%d bit(s), %d word(s)" % (
    len(bits), len(frames))
```

渲染 这三个渲染方法将字显示为十六进制以及其自身位宽的二进制，并标记被截断的字。label() 保持为两位十六进制数，出错时前置一个感叹号，这正是内置解码器所遵循的惯例。

```
def columns(self):
    return [("Time", 92), ("Hex", 60),
            ("Binary", 80), ("Status", 70)]

def row(self, frame, ascii=True):
    return ["0x%02X" % frame.value,
            format(frame.value, "0%db" % frame.length),
            "OK" if frame.status == OK else "CUT"]

def label(self, frame, ascii=True):
    text = "%02X" % (frame.value & 0xFF)
    return ("!" + text) if frame.status else text
```

该文件就是一个完整、可列出的解码器。放入 decoders 文件夹后，它会以 "Pulse-width bits" 的名称出现在选项卡中，带有一个字长框和一个比特序下拉框，并对映射到其唯一输入的任何源进行解码。

串行捷径

不要重新实现比特采样

这个完整示例自行采样比特，因为其协议是自时钟的，且与任何内置协议都不同。对于异步串行族的任何协议，例如 RS-232、RS-422/485、MODBUS、MIDI、LIN、XBee 和 DMX512，请不要重新实现比特采样。调用共享的、经样本集测试的辅助函数 serial_bytes()，它以唯一经过验证的方式完成自动波特率检测、比特中心采样以及奇偶校验位和停止位检查，然后在它返回的字节流上构建帧结构。

```

from tds_decode import Decoder, Param
from tds_decode import serial_bytes, STANDARD_BAUDS

class MyUartDecoder(Decoder):

    NAME = "My UART"
    DESCRIPTION = "Async serial, one wire"
    SOURCES = ["Line"]
    PARAMS = [
        Param("baud", "Bit rate", "choice",
              choices=[(0, "Auto")]
              + [(b, str(b)) for b in STANDARD_BAUDS],
              default=0),
        Param("framing", "Framing", "choice",
              choices=[(f, f) for f in
                       ("8N1", "8E1", "8O1", "8N2")],
              default="8N1"),
    ]

    def decode(self, signals, params):
        sig = signals["Line"]
        frames, err, baud, spb = serial_bytes(
            sig, int(params.get("baud") or 0),
            params.get("framing", "8N1"), inverted=False)
        if err:
            return [], err
        return frames, "%d baud, %.1f smp/bit, %d frames" % (
            baud, spb, len(frames))

```

`serial_bytes(sig, baud, framing, inverted)` 返回四个值：字节帧、成功时为空的错误字符串、检测到的波特率，以及每比特采样数。`baud` 为 0 表示要求它检测速率。`inverted` 为 `True` 时按空闲为低、有效为高的电气极性读取，例如真正的 RS-232。像 MODBUS 或 LIN 这样的更高层协议会调用 `serial_bytes` 取得原始字节，再将它们分组并校验为自己的帧；该模式可参阅 `modbus.py` 和 `lin.py`。

安装与测试

安装

放入并打开

将完成的文件复制到 tdstoolkit.json 旁的可写 decoders 文件夹中，然后打开或重新打开“波形”选项卡。该解码器便会出现在协议列表中。既无注册步骤，也无需重新构建。为它的每个角色映射一个捕获的源，设置其参数，并运行它。

说明. 如果解码器未出现，说明它没有被导入。请查看程序日志中指明该文件和异常的那一行。加载器会跳过无法导入的文件，而不是让整个列表失败，因此有问题的解码器会悄无声息地失效。

测试

用合成 Signal 驱动它

解码器是其 Signal

和参数的纯函数，因此无需仪器即可测试它：从构造的采样数组建立一个 Signal，调用 decode()，并断言结果。构造采样使其含义已知，将其包装进一个 Signal，调用 auto_threshold()——即框架通常会做的那一步——然后检查各帧。

```
from tds_decode import Signal
from pulsebits import PulseBitsDecoder

def _bit(value, period=40):
    """One pulse-width bit: high for part of the period."""
    high = int(period * (0.66 if value else 0.33))
    return [30000] * high + [-30000] * (period - high)

def demo():
    stream = [-30000] * 20 # idle low lead-in
    for bit in (1, 0, 1, 1, 0, 0, 1, 0): # 0xB2, MSB first
        stream += _bit(bit)
    stream += [-30000] * 20 # idle low tail
    sig = Signal(stream, 1e-6) # 1 us per sample
    sig.auto_threshold() # framework does this
    frames, note = PulseBitsDecoder().decode(
        {"Line": sig}, {"bits": 8, "order": "msb"})
    assert len(frames) == 1, note
    assert frames[0].value == 0xB2, "%02X" % frames[0].value
    print("ok:", note)

if __name__ == "__main__":
    demo()
```

测试每个解码器

每个内置解码器都以这种方式检验。RS-232 解码器针对真实捕获的样本集运行；其余每个解码器则针对 `tdstest.py` 中的一个合成向量检验，每个向量在 `bitecheck.py` 中都有一个对应的变异测试项，故意破坏解码器并确认测试能够察觉。每添加一种协议，都应为其添加一个测试用例。



小心. 未经测试的解码终将失准。在依赖其结果之前，请为每个新增的解码器添加一个合成向量，以及一个证明该向量确实起作用的变异测试项。

检查清单

在依赖它之前

- 文件名为小写、不含空格，且不以下划线开头。
- 该类设置了 `NAME`、`DESCRIPTION`、`SOURCES` 和 `PARAMS`，且 `NAME` 不为空。
- `decode()` 在每条路径上都返回 `(frames, note)` 元组，包括失败路径。
- `Frame` 的 `index` 和 `end_index` 均为绝对值：`sig.first_index` 加上局部采样索引。
- 无法读取的记录返回空列表和一条说明原因的注释，绝不抛出异常。
- `columns()`、`row()` 和 `label()` 在字段的数量和顺序上保持一致。
- 合成测试驱动解码器并断言解码得到的值。